



**KHOA CÔNG NGHỆ THÔNG TIN
BỘ MÔN MẠNG VÀ CÁC HỆ THỐNG THÔNG TIN**

CHƯƠNG 5

BỂ TẮC



Nội dung chính - Chương 5

Mục lục

- 5.1. Định nghĩa
- 5.2. Đặc điểm của bế tắc
- 5.3. Ngăn chặn bế tắc
- 5.4. Tránh bế tắc
- 5.5. Phát hiện bế tắc
- 5.6. Khắc phục bế tắc

Từ khóa

Deadlock	Resource graph
Safe state	Safe sequence
Prevention	Avoidance
Banker	Detection
Wait-for graph	Recovery

Trọng tâm kiến thức



Nhận diện bế tắc

Phân biệt bế tắc với chờ bình thường, chết đói và livelock trong hệ thống đồng thời.

Phân tích đồ thị

Dựng đồ thị phân phối tiến trình và suy luận từ chu trình.

Vận dụng thuật toán

Thực hiện Safety, Resource-Request, Banker và Detection trên dữ liệu ma trận.

Câu hỏi định hướng: Hệ điều hành nên ngăn bế tắc từ trước, tránh trạng thái nguy hiểm hay cho phép rồi khắc phục?



5.1. Định nghĩa

Bế tắc xuất hiện khi một nhóm tiến trình chờ lẫn nhau và không thành viên nào tự tạo được sự kiện giải phóng



Bế tắc là trạng thái chờ khép kín

Một tập tiến trình

Xét một nhóm tiến trình đang hoạt động trong hệ thống và có quan hệ phụ thuộc tài nguyên.

Mỗi tiến trình bị chặn

Mỗi thành viên giữ ít nhất một tài nguyên hoặc trạng thái, đồng thời chờ thêm tài nguyên.

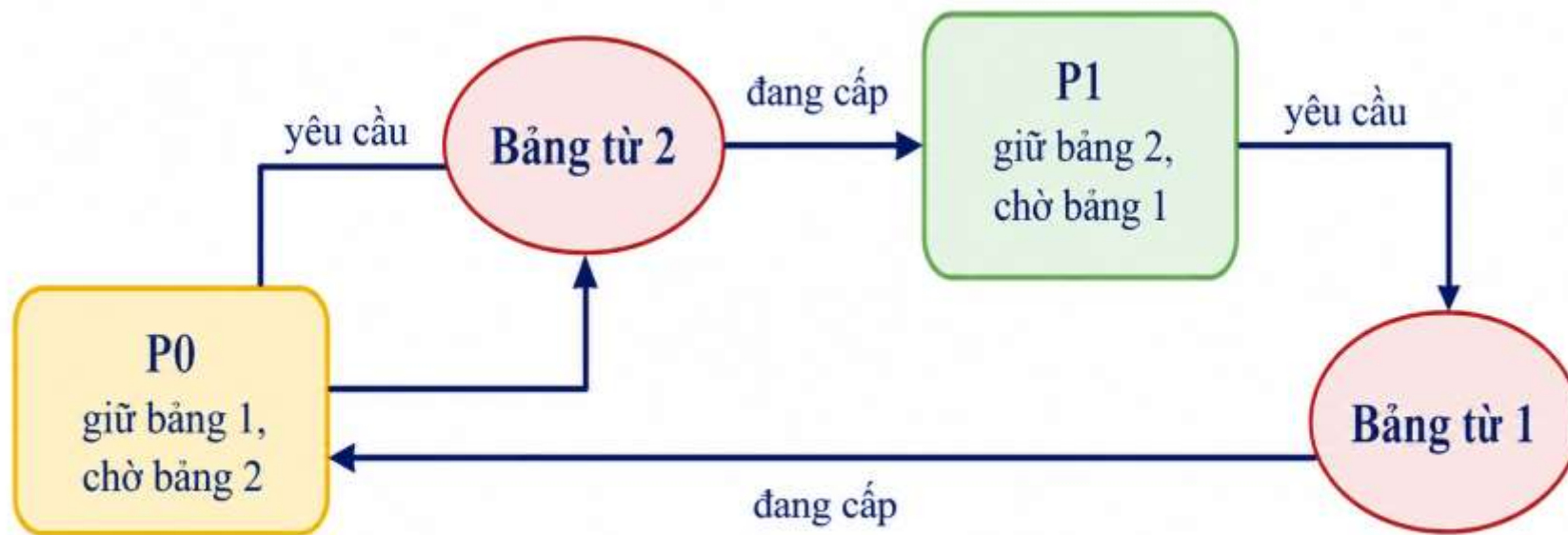
Không ai tự tiến triển

Tài nguyên cần thiết chỉ có thể được giải phóng bởi một tiến trình khác cũng đang bị chặn trong nhóm.

Bế tắc là thuộc tính của cả một tập tiến trình và quan hệ chờ giữa chúng, không chỉ của một tiến trình đơn lẻ.



Hai tiến trình có thể khóa lẫn nhau



P0 giữ bảng từ 1 và chờ bảng từ 2; P1 giữ bảng từ 2 và chờ bảng từ 1. Không tiến trình nào có thể tự hoàn thành để trả tài nguyên.



Mô hình hệ thống gồm loại tài nguyên và các thể hiện

Loại tài nguyên

CPU, vùng nhớ, tệp, khóa, thiết bị vào/ra hoặc một cấu trúc dữ liệu cần truy cập độc quyền.

Số thể hiện

Một loại có thể chỉ có một thể hiện hoặc nhiều thể hiện tương đương được cấp cho các tiến trình.

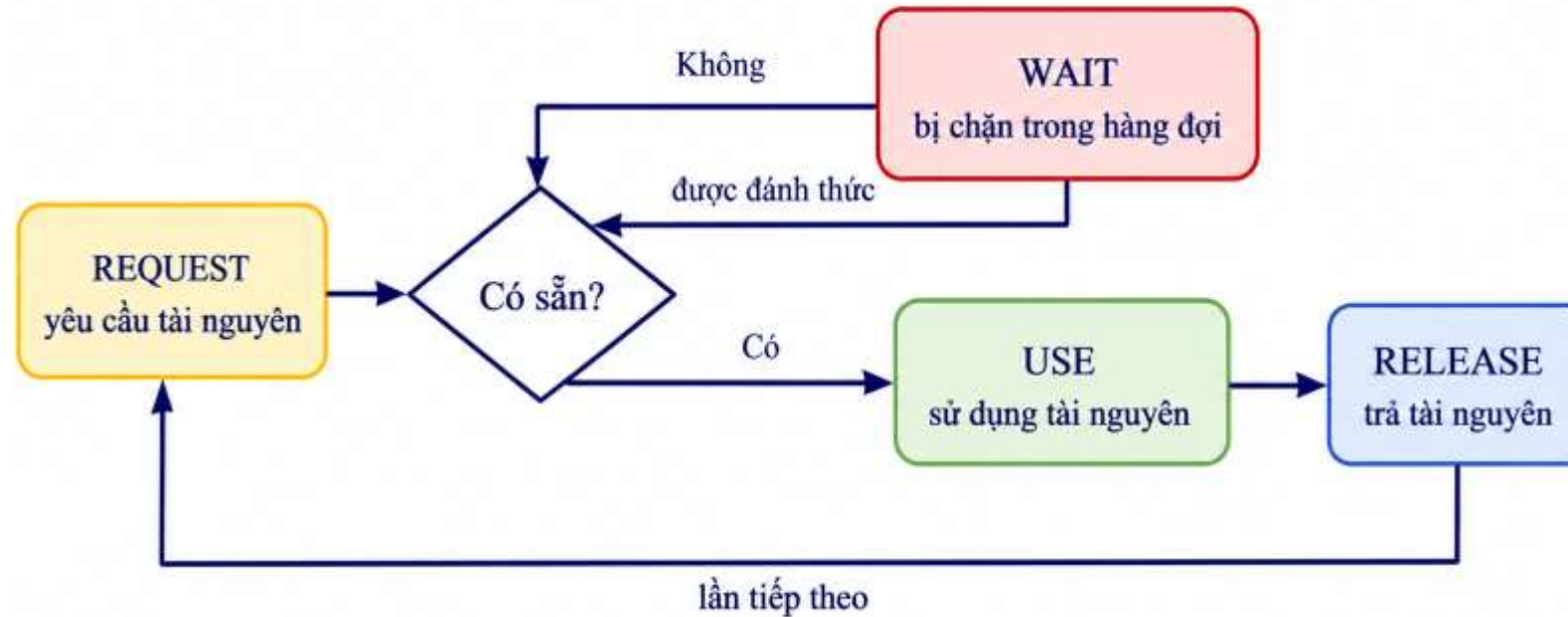
Trạng thái phân phối

Hệ điều hành theo dõi số còn rồi, số đang cấp và các yêu cầu chưa được thỏa mãn.

Bế tắc liên quan đến tài nguyên không thể được chia sẻ tùy ý hoặc không thể bị thu hồi an toàn tại mọi thời điểm.



Một tiến trình sử dụng tài nguyên qua ba bước



Request có thể được cấp ngay hoặc làm tiến trình chờ; sau khi use, tiến trình phải release để tài nguyên quay lại hệ thống.



Không phải mọi tình trạng chờ đều là bế tắc

Chờ bình thường

Tiến trình bị chặn nhưng tài nguyên sẽ được trả bởi một tiến trình vẫn đang tiến triển.

Bế tắc

Một nhóm tiến trình chờ khép kín; không thành viên nào có thể tự giải phóng điều kiện chờ.

Chết đói

Hệ thống vẫn tiến triển nhưng một tiến trình liên tục bị vượt qua và không nhận được tài nguyên.

Livelock

Các tiến trình vẫn đổi trạng thái hoặc nhường nhau nhưng không tạo ra tiến triển hữu ích.



Hệ điều hành có bốn hướng xử lý bế tắc

Ngăn chặn

Ràng buộc cách yêu cầu tài nguyên để ít nhất một điều kiện cần không thể xảy ra.

Tránh

Biết nhu cầu cực đại và chỉ cấp phát khi trạng thái sau cấp vẫn an toàn.

Phát hiện và khắc phục

Cho phép bế tắc, định kỳ phát hiện rồi kết thúc hoặc rollback tiến trình.

Bỏ qua

Giả định bế tắc hiếm và để ứng dụng/người dùng xử lý; còn gọi là cách tiếp cận ostrich.



5.2. Đặc điểm của bế tắc

Bế tắc chỉ có thể xuất hiện khi bốn điều kiện cần đồng thời đúng và quan hệ chờ tạo thành cấu trúc khép kín



Bốn điều kiện cần phải đồng thời tồn tại

Mutual exclusion (Độc quyền truy xuất)

Ít nhất một tài nguyên chỉ phục vụ một tiến trình tại một thời điểm.

Hold and wait (Giữ và chờ)

Tiến trình vừa giữ tài nguyên đã cấp, vừa chờ tài nguyên khác.

No preemption (Không chiếm đoạt)

Tài nguyên không bị hệ thống thu hồi cưỡng bức; chủ sở hữu tự giải phóng.

Circular wait (Chờ vòng tròn)

Tồn tại chuỗi P_0 chờ P_1 , P_1 chờ P_2 , ..., P_n chờ P_0 .



Độc quyền truy xuất làm tài nguyên không thể chia sẻ

Ý nghĩa

Tại một thời điểm, một thể hiện tài nguyên chỉ được một tiến trình sử dụng; tiến trình khác phải đợi.

Ví dụ

Máy in, mutex đang khóa, tệp đang cập nhật hoặc cấu trúc dữ liệu cần bảo vệ bất biến.

Tài nguyên chỉ đọc hoặc có thể sao chép an toàn thường không cần độc quyền; khi đó điều kiện này có thể bị loại bỏ.



Giữ và chờ kết hợp quyền sở hữu với yêu cầu mới

Đang giữ

Tiến trình chưa giải phóng các tài nguyên đã được cấp, vì chúng còn cần thiết cho công việc đang thực hiện.

Đang chờ

Cùng lúc đó, tiến trình yêu cầu thêm một hoặc nhiều tài nguyên hiện do tiến trình khác giữ.

Nếu mọi yêu cầu phải được cấp cùng lúc hoặc tiến trình chỉ được xin khi không giữ gì, điều kiện hold-and-wait không tồn tại.



Không chiếm đoạt khiến hệ thống phải chờ chủ sở hữu

Quy tắc

Tài nguyên chỉ được giải phóng tự nguyện sau khi tiến trình hoàn thành đoạn công việc liên quan.

Lý do

Thu hồi giữa chừng có thể làm mất trạng thái, phá bất biến hoặc khiến thao tác vào/ra không thể khôi phục.

CPU và trang nhớ có thể bị thu hồi hoặc hoán đổi; máy in đang in hay mutex bảo vệ cập nhật thường khó bị chiếm đoạt an toàn.



Chờ vòng tròn đóng kín chuỗi phụ thuộc

Chuỗi chờ

P0 chờ tài nguyên do P1 giữ; P1 chờ P2; tiếp tục cho đến Pn chờ tài nguyên do P0 giữ.

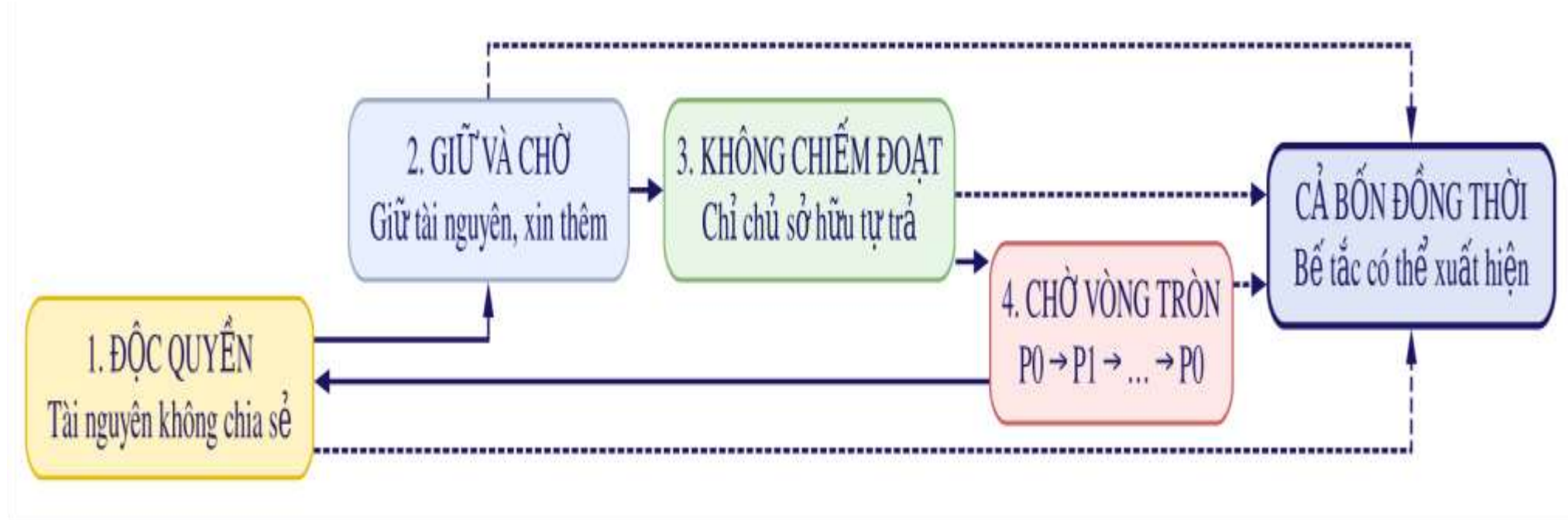
Hệ quả

Mọi tiến trình trong vòng chỉ có thể tiếp tục sau một tiến trình khác trong chính vòng đó, nên không có điểm bắt đầu.

Circular wait là biểu hiện cấu trúc của bế tắc; áp đặt thứ tự tài nguyên toàn cục là cách trực tiếp để phá điều kiện này.



Bốn điều kiện kết hợp mới tạo nguy cơ bế tắc



Mỗi điều kiện là cần nhưng chưa đủ. Chỉ cần thiết kế làm một điều kiện luôn sai thì bế tắc bị ngăn chặn về mặt cấu trúc.



Đồ thị cấp phát tài nguyên mô tả quan hệ giữ và chờ

Nút tiến trình

Hình tròn P_i biểu diễn một tiến trình đang tham gia cấp phát hoặc yêu cầu tài nguyên.



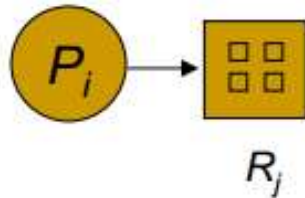
Nút tài nguyên

Hình vuông R_j : Các chấm bên trong biểu diễn số thể hiện của loại tài nguyên.



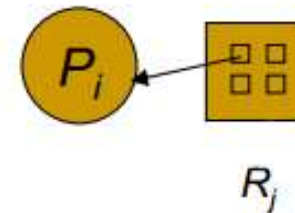
Cạnh yêu cầu

$P_i \rightarrow R_j$: P_i đang chờ một thể hiện của R_j .

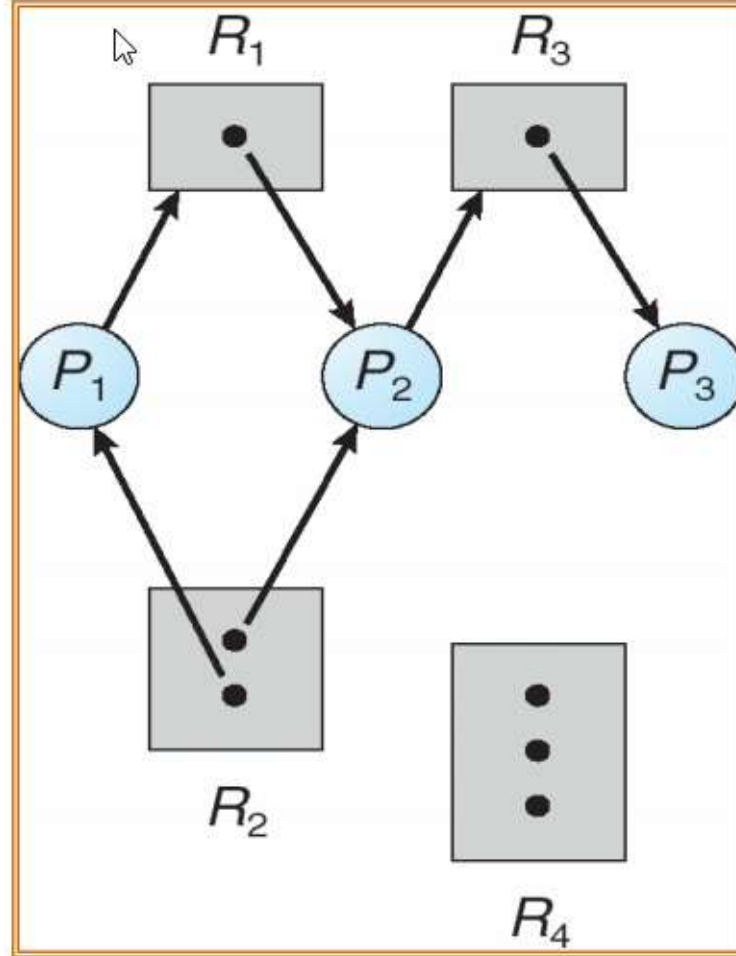


Cạnh phân phối

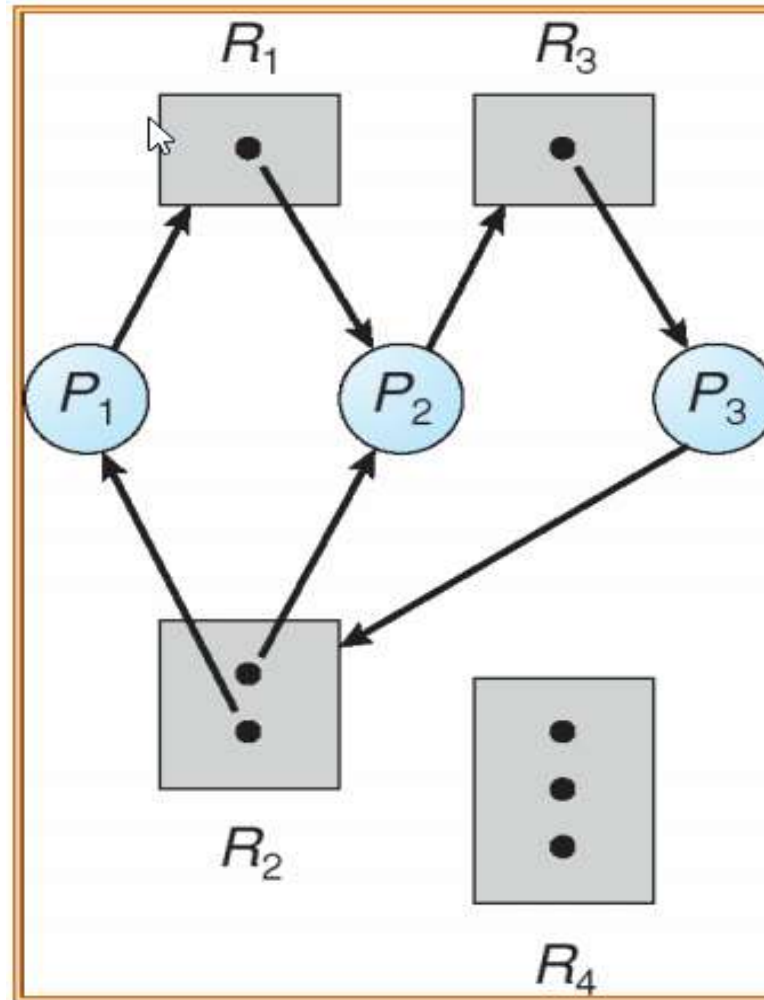
$R_j \rightarrow P_i$: một thể hiện của R_j đang được cấp cho P_i .



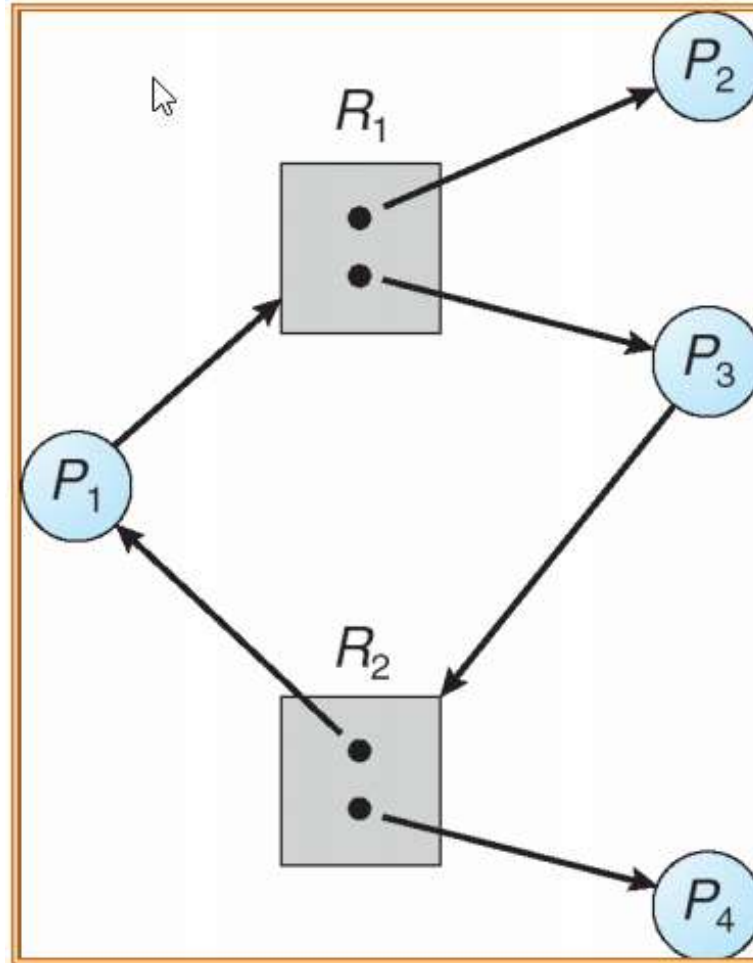
Không có chu trình thì không có bế tắc



Một thể hiện mỗi loại: Chu trình đồng nghĩa bề tắc



Nhiều thể hiện: Có chu trình chưa chắc đã bế tắc



Tiên đề từ đồ thị phân phối tài nguyên

Không chu trình

Hệ thống chắc chắn không bế tắc đối với trạng thái cấp phát đang xét.

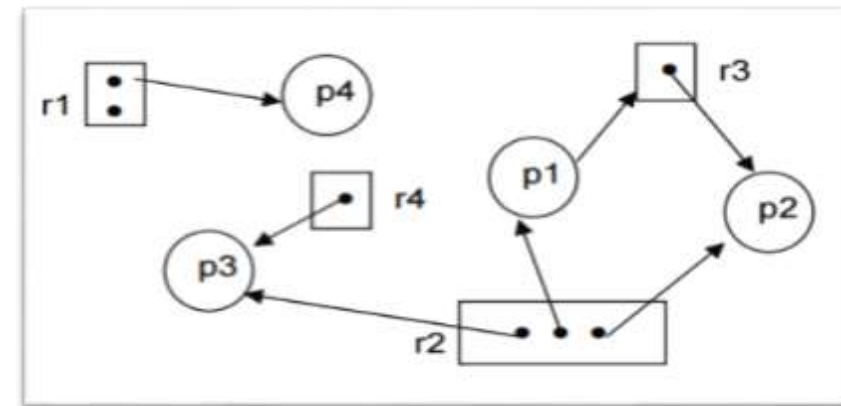
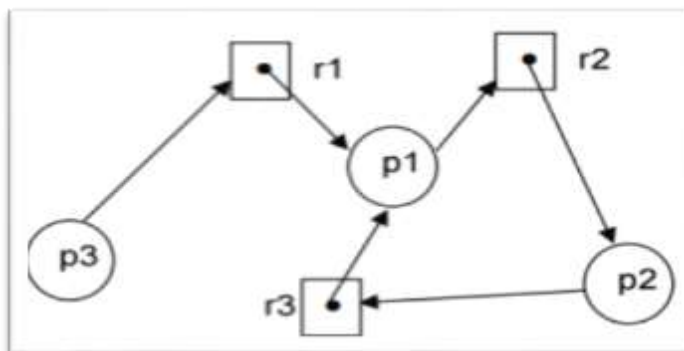
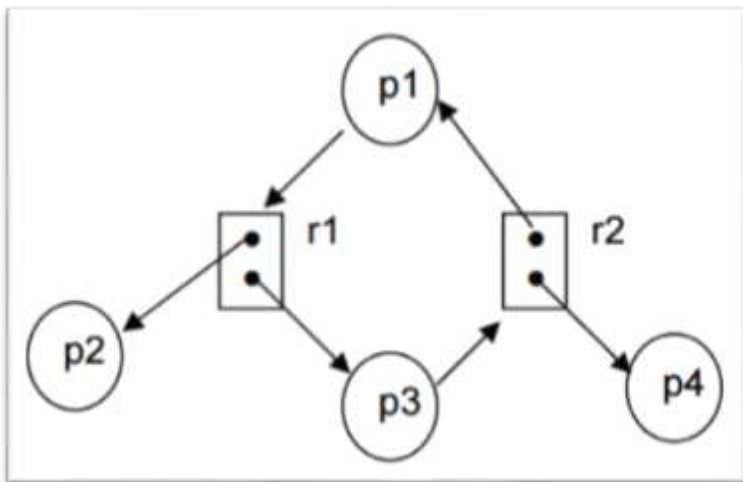
Có chu trình, một thể hiện

Chu trình là điều kiện cần và đủ; các tiến trình trong chu trình đang bế tắc.

Có chu trình, nhiều thể hiện

Chu trình chỉ là điều kiện cần; phải xét số thể hiện rồi và khả năng tiến trình ngoài vòng hoàn thành.

“Đồ thị giúp nhìn cấu trúc phụ thuộc; với nhiều thể hiện, thuật toán ma trận cung cấp kết luận chính xác hơn.”





5.3. Ngăn chặn bế tắc

Ngăn chặn biến một điều kiện cần thành bất khả thi bằng các quy tắc cấp phát cố định



Ít nhất một điều kiện cần để ngăn chặn bế tắc

Thiết kế trước

Quy tắc được áp dụng cho mọi tiến trình, không cần dự đoán chuỗi yêu cầu cụ thể tại thời gian chạy.

Bảo đảm mạnh

Nếu một trong bốn điều kiện không thể đúng, hệ thống không thể đi vào bế tắc.

Đổi bằng hiệu năng

Hạn chế yêu cầu có thể làm giảm mức sử dụng tài nguyên, giảm tính đồng thời hoặc gây chết đói.

Ngăn chặn bế tắc dễ áp dụng hơn, nhưng thường hạn chế tài nguyên nhiều hơn so với tránh bế tắc



Ưu tiên dùng chung tài nguyên nếu có thể

Có thể áp dụng

Dữ liệu bất biến, tệp chỉ đọc, thư viện dùng chung hoặc kết quả được sao chép có thể phục vụ nhiều tiến trình.

Không thể áp dụng

Máy in, thiết bị ghi, khóa cập nhật hoặc tài nguyên có trạng thái thay đổi vẫn cần độc quyền để giữ tính đúng đắn.

Không thể loại bỏ mutual exclusion cho mọi tài nguyên; vì vậy chiến lược này chỉ giảm phạm vi rủi ro chứ không phải lời giải phổ quát.



Yêu cầu tất cả cùng lúc để phá điều kiện giữ và chờ (Hold & Wait)

Phương án 1

Tiến trình khai báo và nhận toàn bộ tài nguyên trước khi bắt đầu; nếu thiếu một tài nguyên thì chưa giữ gì cả.

Phương án 2

Chỉ cho phép yêu cầu mới khi tiến trình không giữ tài nguyên; muốn xin thêm phải trả toàn bộ rồi yêu cầu lại.

Nhược điểm: Tài nguyên bị giữ lâu nhưng chưa dùng, mức sử dụng thấp; tiến trình cần bộ tài nguyên lớn có thể dẫn tới chết đói.



Cho phép thu hồi để phá điều kiện không chiếm đoạt

Khi yêu cầu không được cấp

Tiến trình phải trả các tài nguyên đang giữ; chúng được đưa vào danh sách chờ cùng tài nguyên mới.

Chạy lại

Tiến trình chỉ tiếp tục khi có thể nhận lại tài nguyên cũ và tài nguyên mới, hoặc khôi phục từ trạng thái nhất quán.

Phù hợp với CPU, thanh ghi hoặc trang nhớ; khó áp dụng cho thiết bị và thao tác có hiệu ứng bên ngoài không thể đảo ngược.



Tạo thứ tự tài nguyên phá điều kiện chờ vòng tròn

Đánh số

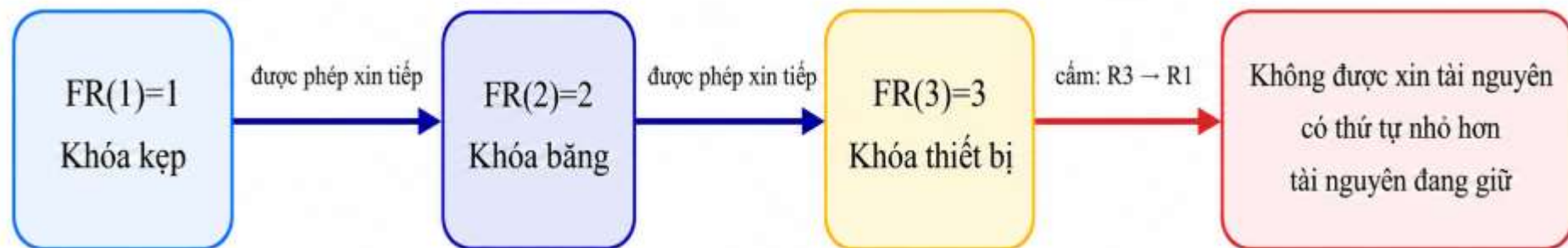
Gán hàm thứ tự $F(R)$ cho mọi loại tài nguyên; tiến trình chỉ được yêu cầu theo thứ tự tăng dần.

Quy tắc

Nếu đang giữ R_i , tiến trình chỉ được xin R_j khi $F(R_j) > F(R_i)$; mọi cạnh chờ đều đi theo một hướng.

Một chu trình sẽ đòi hỏi thứ tự tăng nghiêm ngặt quay về điểm ban đầu, điều này không thể xảy ra.

Thứ tự tăng loại bỏ cạnh chờ quay ngược



Tất cả mã phải tuân cùng thứ tự. Quy tắc đơn giản nhưng việc chọn thứ tự phù hợp cho hệ thống lớn có thể làm giảm tính linh hoạt.



Mỗi phương pháp ngăn chặn có một nhược điểm khác nhau

Chia sẻ

Tăng tính đồng thời nhưng chỉ hợp lệ với tài nguyên thực sự có thể chia sẻ an toàn.

Xin tất cả

Dễ bảo đảm nhưng giữ tài nguyên chưa dùng, giảm hiệu suất và có thể gây chết đói.

Chiếm đoạt

Giải phóng tài nguyên linh hoạt nhưng cần rollback hoặc trạng thái có thể tái tạo.

Thứ tự toàn cục

Phổ biến cho khóa phần mềm; có thể làm mã phức tạp và buộc lấy khóa sớm hơn cần thiết.



5.4. Tránh bế tắc

Tránh bế tắc quyết định từng yêu cầu dựa trên khả năng hệ thống vẫn hoàn thành mọi tiến trình



Tránh bế tắc giữ hệ thống trong miền an toàn

Thông tin trước

Mỗi tiến trình khai báo nhu cầu tối đa cho từng loại tài nguyên hoặc các cạnh claim có thể phát sinh.

Kiểm tra trước khi cấp

Hệ điều hành mô phỏng trạng thái sau cấp và tìm một thứ tự để mọi tiến trình lần lượt hoàn thành.

Trì hoãn khi cần

Yêu cầu hợp lệ và tài nguyên đang rỗi vẫn có thể bị hoãn nếu cấp ngay làm trạng thái không an toàn.

Không an toàn không đồng nghĩa đã bế tắc; nó nghĩa là hệ thống không còn bảo đảm tránh bế tắc với mọi yêu cầu tối đa hợp lệ.



An toàn, không an toàn và bế tắc là ba trạng thái khác nhau



Mọi trạng thái bế tắc đều không an toàn, nhưng một trạng thái không an toàn có thể chưa bế tắc. Thuật toán tránh không cho hệ thống rời miền an toàn.



Hệ thống an toàn nếu tồn tại 1 chuỗi an toàn

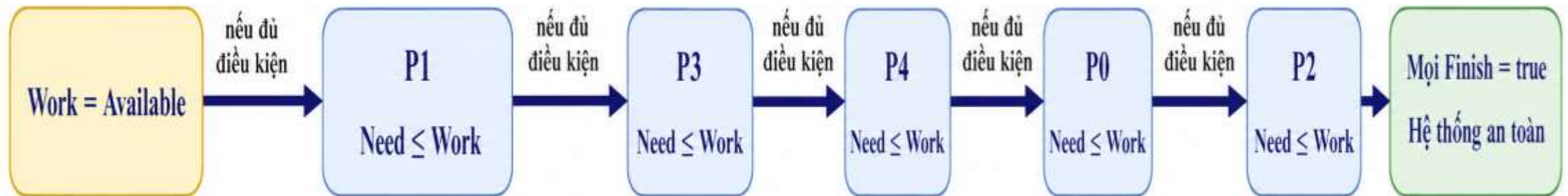
- Xét hệ thống có 12 tài nguyên là 12 băng từ
 - Tại thời điểm t_0 hệ thống ở trạng thái như sau:

Process	Max needs	Allocated	Current needs
P0	10	5	5
P1	4	2	2
P2	9	2	7

- Tại thời điểm t_1 , P2 xin cấp phát 1 tài nguyên và được chấp nhận. Hệ thống chuyển sang trạng thái như sau:

Process	Max needs	Allocated	Current needs
P0	10	5	5
P1	4	2	2
P2	9	3	6

Chuỗi an toàn chứng minh khả năng hoàn thành



Nếu Need của tiến trình kế tiếp không vượt Work, giả sử tiến trình hoàn thành và cộng Allocation của nó vào Work. Lặp đến khi mọi Finish = true.



Cạnh claim mô tả yêu cầu có thể xuất hiện



Với mỗi loại chỉ có một thể hiện, cạnh claim nét đứt chuyển thành request khi tiến trình yêu cầu; chỉ chuyển request thành assignment nếu không tạo chu trình.



Khi nào dùng thuật toán đồ thị cấp phát tài nguyên?

Tránh bế tắc – Trường hợp đơn giản

Điều kiện áp dụng

Mỗi loại tài nguyên chỉ có một thể hiện.

Ví dụ: mỗi loại chỉ có một máy in, một thiết bị hoặc một khóa.

Thông tin cần biết

Tiến trình phải khai báo trước những tài nguyên có thể yêu cầu.

Thông tin này được biểu diễn bằng cạnh claim (nét đứt).

Quy tắc cấp phát

Chỉ cấp tài nguyên khi việc đổi cạnh yêu cầu thành cạnh cấp phát không tạo chu trình.

Không tạo chu trình → trạng thái vẫn an toàn. Tạo chu trình → yêu cầu phải chờ.



Thuật toán đồ thị cấp phát tài nguyên: Ba bước

1

Tiến trình gửi yêu cầu

Cạnh claim $P_i \rightsquigarrow R_j$ được đổi thành cạnh yêu cầu $P_i \rightarrow R_j$.

2

Thử cấp tài nguyên

Tạm đổi cạnh yêu cầu thành cạnh cấp phát $R_j \rightarrow P_i$.

3

Kiểm tra chu trình

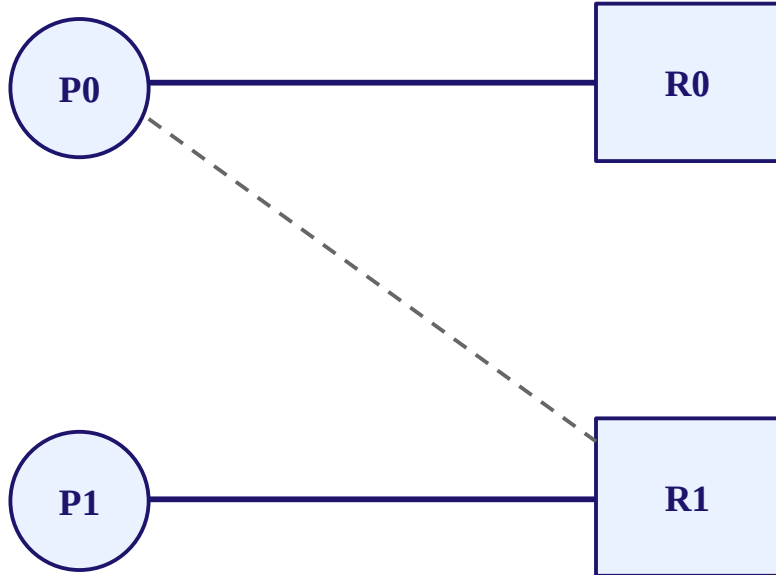
Không có chu trình thì cấp thật; có chu trình thì khôi phục và cho tiến trình chờ.

Ý tưởng chính: thử trước trên mô hình, chỉ cấp khi kết quả vẫn an toàn.



Ví dụ: Yêu cầu được cấp vì không tạo chu trình

Trạng thái trước khi cấp



R0 đang cấp cho P0

P1 yêu cầu R1

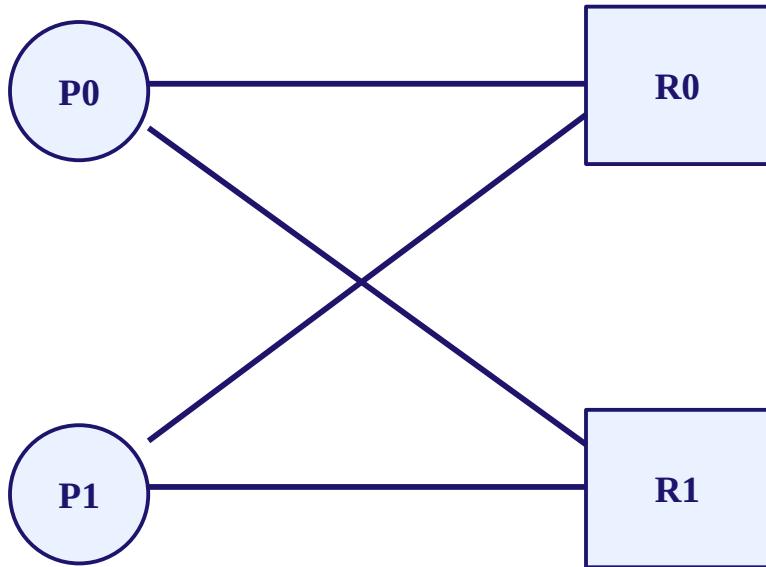
Cấp R1 cho P1 không tạo chu trình.

Kết luận: yêu cầu được chấp nhận.



Ví dụ: Yêu cầu phải chờ vì tạo chu trình

Thử cấp tài nguyên



P0 chờ R1

P1 chờ R0

$P0 \rightarrow R1 \rightarrow P1 \rightarrow R0 \rightarrow P0$

Đã xuất hiện chu trình.

Kết luận: không cấp ngay, tiến trình phải chờ.



Thuật toán Banker: Hãy hình dung như một ngân hàng

Tránh bế tắc – Nhiều thể hiện tài nguyên

Ngân hàng

Ngân hàng không cho vay hết tiền nếu điều đó có thể khiến một khách hàng không bao giờ nhận đủ khoản cần thiết.

Hệ điều hành

Hệ điều hành không cấp hết tài nguyên nếu sau đó không còn cách bảo đảm mọi tiến trình hoàn thành.

Kết quả

Yêu cầu có thể hợp lệ và tài nguyên đang rỗi, nhưng vẫn bị hoãn nếu làm hệ thống mất an toàn.

Banker không đợi bế tắc xảy ra; thuật toán kiểm tra trước khi cấp.



Banker cần theo dõi những dữ liệu nào?

Available

Số tài nguyên hiện đang rỗi của từng loại.

Ví dụ: (3, 3, 2)

Max

Số tài nguyên tối đa mỗi tiến trình có thể cần.

Được khai báo trước.

Allocation

Số tài nguyên hiện đã cấp cho từng tiến trình.

Need

Số tài nguyên tiến trình còn có thể cần thêm.

$\text{Need} = \text{Max} - \text{Allocation}$

Mỗi hàng tương ứng một tiến trình; mỗi cột tương ứng một loại tài nguyên.



Cách tính ma trận Need

Trong ví dụ sau, ba loại tài nguyên được ký hiệu là A, B và C.

Công thức

$$\text{Need}[i, j] = \text{Max}[i, j] - \text{Allocation}[i, j]$$

Ví dụ với tiến trình P1

Max của P1

3	2	2
---	---	---

Allocation của P1

2	0	0
---	---	---

=

Need của P1

1	2	2
---	---	---

Tiến trình	A	B	C
P0	7	4	3
P1	1	2	2
P2	6	0	0
P3	0	1	1
P4	4	3	1

Need cho biết giới hạn yêu cầu còn lại; không phải yêu cầu đang chờ ở thời điểm hiện tại.



Ví dụ Banker: Dữ liệu ban đầu

Allocation – Đã cấp

Tiến trình	A	B	C
P0	0	1	0
P1	2	0	0
P2	3	0	2
P3	2	1	1
P4	0	0	2

Max – Nhu cầu tối đa

Tiến trình	A	B	C
P0	7	5	3
P1	3	2	2
P2	9	0	2
P3	2	2	2
P4	4	3	3

Available

3	3	2
---	---	---



Thuật toán Safety trả lời câu hỏi gì?

Câu hỏi cần trả lời

“Với số tài nguyên đang rồi, có thể chọn một tiến trình hoàn thành trước, rồi tiếp tục cho đến khi tất cả hoàn thành không?”

Có

Tồn tại một chuỗi an toàn → trạng thái an toàn.

Không

Không tìm được chuỗi an toàn → trạng thái không an toàn.

Safety chỉ kiểm tra trạng thái; Banker dùng kết quả này để quyết định có cấp yêu cầu hay không.



Thuật toán Safety: Thực hiện theo bốn bước

1 Khởi tạo

$Work = Available$. Đánh dấu tất cả tiến trình là chưa hoàn thành.

2 Tìm tiến trình phù hợp

Chọn P_i chưa hoàn thành và $Need[i] \leq Work$.

3 Giả sử tiến trình hoàn thành

$Work = Work + Allocation[i]$; đánh dấu P_i đã hoàn thành.

4 Lặp lại và kết luận

Nếu mọi tiến trình hoàn thành $\rightarrow$ an toàn; nếu không tìm được $P_i \rightarrow$ không an toàn.

So sánh $Need \leq Work$ được thực hiện theo từng cột A, B, C.



Safety – Bước đầu: Work = Available

Work ban đầu

3	3	2
---	---	---

Finish = false cho P0, P1, P2, P3, P4

Tiến trình	Need A	Need B	Need C
P0	7	4	3
P1	1	2	2
P2	6	0	0
P3	0	1	1
P4	4	3	1

Kiểm tra P1

Need(P1) = (1, 2, 2)
Work = (3, 3, 2)
 $1 \leq 3, 2 \leq 3, 2 \leq 2$

P1 có thể hoàn thành trước.



Safety – P1 hoàn thành và trả tài nguyên

Work trước				Allocation(P1)				Work mới		
3	3	2	+	2	0	0	=	5	3	2

P1 hoàn thành $\rightarrow$ Finish(P1) = true

Tiếp theo, $\text{Need}(P3) = (0, 1, 1) \leq \text{Work} = (5, 3, 2)$

Chọn P3 là tiến trình tiếp theo.



Safety – P3 hoàn thành, Work tiếp tục tăng

Work trước				Allocation(P3)				Work mới		
5	3	2	+	2	1	1	=	7	4	3

P3 hoàn thành $\rightarrow$ Finish(P3) = true

P4

Need = $(4, 3, 1) \leq (7, 4, 3)$

P0

Sau khi P4 trả tài nguyên, P0 có thể hoàn thành.

P2

Cuối cùng P2 nhận đủ tài nguyên và hoàn thành.

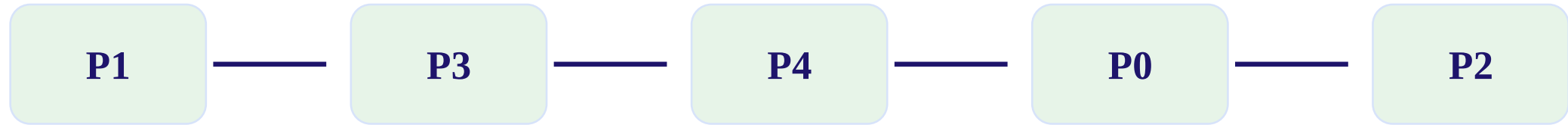
Safety – Hoàn thành các tiến trình còn lại



Thời điểm	Work	Tiến trình có thể chọn tiếp
Sau P1	(5, 3, 2)	P3
Sau P3	(7, 4, 3)	P4
Sau P4	(7, 4, 5)	P0
Sau P0	(7, 5, 5)	P2
Sau P2	(10, 5, 7)	Hoàn tất

Mỗi tiến trình hoàn thành sẽ trả Allocation, vì vậy Work thường tăng dần.

Kết luận Safety: Tìm được một chuỗi an toàn



Chuỗi an toàn: $\langle P1, P3, P4, P0, P2 \rangle$

Vì mọi tiến trình đều có thể lần lượt hoàn thành, trạng thái ban đầu là an toàn.



Thuật toán yêu cầu tài nguyên: Ba lần kiểm tra

1

Không xin quá nhu cầu đã khai báo
Nếu $\text{Request}[i] > \text{Need}[i]$ thì yêu cầu không hợp lệ.

2

Tài nguyên phải đang có sẵn
Nếu $\text{Request}[i] > \text{Available}$ thì tiến trình phải chờ.

3

Cấp thử rồi chạy Safety
Nếu trạng thái sau cấp vẫn an toàn thì cấp thật; nếu không thì hoàn tác.

Hai kiểm tra đầu chỉ cho biết yêu cầu hợp lệ; Safety mới quyết định có nên cấp ngay hay không.



Ví dụ: P1 yêu cầu thêm (1, 0, 2)

Request(P1)

1	0	2
---	---	---

Need(P1)

1	2	2
---	---	---

Available

3	3	2
---	---	---

Kiểm tra 1

$(1, 0, 2) \leq (1, 2, 2) \rightarrow \text{Đúng}$

Kiểm tra 2

$(1, 0, 2) \leq (3, 3, 2) \rightarrow \text{Đúng}$

Yêu cầu hợp lệ và tài nguyên đang đủ. Bước tiếp theo: cấp thử và kiểm tra Safety.



Cấp thử cho P1: Các giá trị thay đổi

Available mới

$$(3, 3, 2) - (1, 0, 2) = (2, 3, 0)$$

Allocation(P1) mới

$$(2, 0, 0) + (1, 0, 2) = (3, 0, 2)$$

Need(P1) mới

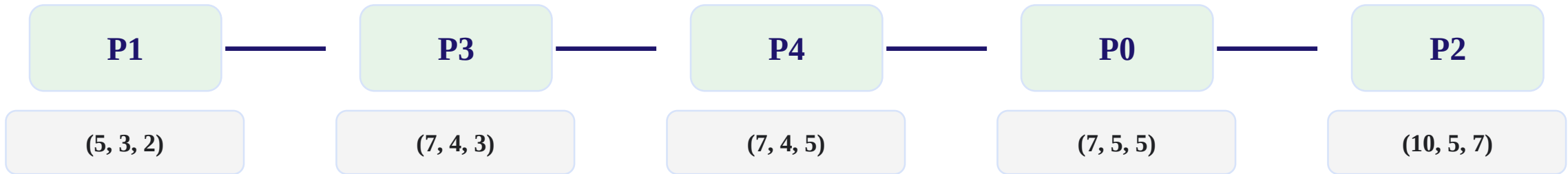
$$(1, 2, 2) - (1, 0, 2) = (0, 2, 0)$$

Sau khi cập nhật tạm thời, chạy lại thuật toán Safety với $Work = (2, 3, 0)$.

Mọi thay đổi ở bước này chỉ là “cấp thử”; nếu trạng thái không an toàn, hệ thống sẽ khôi phục dữ liệu cũ.



Chạy Safety sau khi cấp thử cho P1



Vẫn tìm được chuỗi an toàn $\langle P1, P3, P4, P0, P2 \rangle$

Kết luận: cấp thật yêu cầu (1, 0, 2) cho P1.



Ví dụ: Vì sao P0 yêu cầu (0, 2, 0) phải chờ?

Xét sau khi P1 đã được cấp (1, 0, 2)

Available hiện tại			Request(P0)			Need(P0)		
2	3	0	0	2	0	7	4	3

Hai kiểm tra đầu đều đúng, nên hệ thống cấp thử.

Work sau cấp thử = (2, 1, 0)

Không có tiến trình nào có $\text{Need} \leq (2, 1, 0)$

Trạng thái sau cấp thử không an toàn $\rightarrow$ hoàn tác và cho P0 chờ.



Tóm tắt các thuật toán dùng để tránh bế tắc

Đồ thị cấp phát tài nguyên

Dùng khi mỗi loại tài nguyên chỉ có một thể hiện.

Cấp yêu cầu nếu không tạo chu trình.

Safety

Kiểm tra một trạng thái có an toàn hay không bằng cách tìm chuỗi hoàn thành.

Resource-Request / Banker

Cấp thử yêu cầu, chạy Safety, rồi quyết định cấp thật hoặc cho chờ.

Safety là phép kiểm tra; Banker là quy trình ra quyết định cấp tài nguyên.



5.5. Phát hiện bế tắc

Phát hiện cho phép cấp phát linh hoạt, sau đó tìm chu trình hoặc tiến trình không thể hoàn thành



Một thể hiện mỗi loại: Dừng đồ thị chờ (Wait-for graph)

Nút

Mỗi nút biểu diễn một tiến trình.
Không cần vẽ nút tài nguyên.

Cạnh $P_i \rightarrow P_j$

P_i đang chờ một tài nguyên do P_j giữ.

Kết luận

Đồ thị có chu trình $\rightarrow$ các tiến trình trong chu trình bị bế tắc.

Wait-for graph phù hợp khi mỗi loại tài nguyên chỉ có một thể hiện.

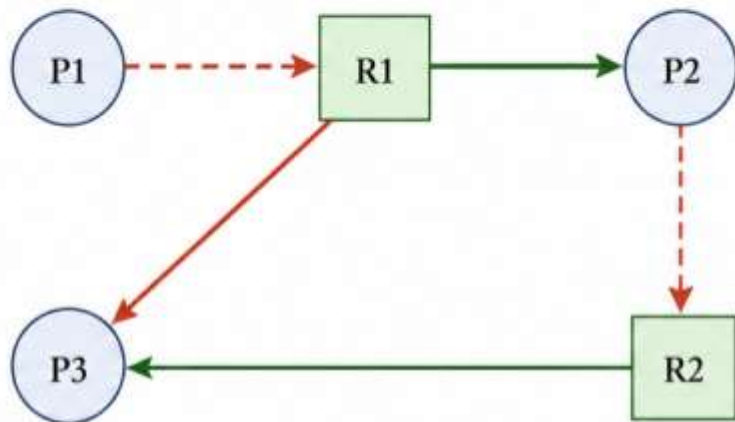
Từ đồ thị cấp phát sang đồ thị chờ



1. ĐỒ THỊ CẤP PHÁT (Resource Allocation Graph)

Biểu diễn trạng thái cấp phát tài nguyên cho các tiến trình.

- Hình tròn: tiến trình (P)
- Hình vuông: tài nguyên (R)



- Cạnh cấp phát ($R \rightarrow P$): tài nguyên đã cấp cho tiến trình
- -> Cạnh yêu cầu ($P \rightarrow R$): tiến trình đang yêu cầu tài nguyên

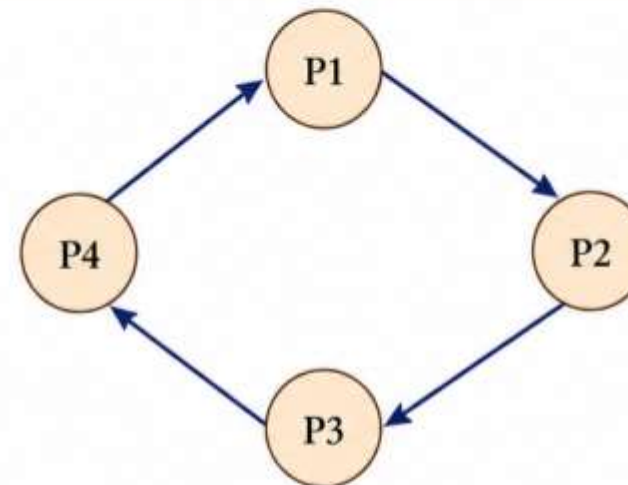
Ví dụ:

- R1 đang cấp cho P2.
- P1 đang yêu cầu R1.
- P2 đang yêu cầu R2.
- R2 đang cấp cho P3.

2. ĐỒ THỊ CHỜ (Wait-for Graph)

Đồ thị chỉ gồm các tiến trình.

Có cạnh từ P_i đến P_j nếu P_i đang chờ một tài nguyên đang được P_j giữ.



- Cạnh chờ ($P_i \rightarrow P_j$): P_i đang chờ tài nguyên do P_j giữ

Ví dụ:

- P1 đang chờ tài nguyên do P2 giữ.
- P2 đang chờ tài nguyên do P3 giữ.
- P3 đang chờ tài nguyên do P4 giữ.
- P4 đang chờ tài nguyên do P1 giữ.

Kết luận:

Nếu đồ thị chờ có chu trình, hệ thống đang ở trạng thái bế tắc.



Nhiều thể hiện: Dùng thuật toán Detection

Available

Số tài nguyên đang rỗi của từng loại.

Allocation

Số tài nguyên đang cấp cho mỗi tiến trình.

Request

Số tài nguyên mỗi tiến trình đang chờ ngay lúc kiểm tra.

Khác Banker: Detection dùng Request hiện tại, không dùng Max và Need tối đa.



Thuật toán Detection: Các bước thực hiện

1 Khởi tạo

Work = Available. Tiến trình không giữ tài nguyên được xem là có thể kết thúc.

2 Tìm tiến trình có thể tiếp tục

Chọn P_i chưa kết thúc và $\text{Request}[i] \leq \text{Work}$.

3 Giả sử P_i hoàn thành

Work = Work + Allocation[i]; đánh dấu P_i đã kết thúc.

4 Kết luận

Sau khi không tìm thêm được P_i , tiến trình còn lại chính là nhóm bế tắc.

Detection và Safety có cách lặp tương tự, nhưng sử dụng dữ liệu khác nhau.



Ví dụ Detection: Xác định tiến trình bị bế tắc

Available

0	0
---	---

Allocation

Tiến trình	A	B
P0	1	0
P1	0	1
P2	0	0

Request hiện tại

Tiến trình	A	B
P0	0	1
P1	1	0
P2	0	0

P2 kết thúc nhưng không trả thêm tài nguyên. P0 cần B, P1 cần A; cả hai đều không thể tiếp tục.

Kết luận: P0 và P1 đang bế tắc.



5.6. Khắc phục bế tắc

Khắc phục phá vòng chờ bằng cách loại bỏ tiến trình hoặc thu hồi tài nguyên, đồng thời kiểm soát chặt chẽ công việc



Kết thúc tiến trình giải phóng tài nguyên ngay

Dừng toàn bộ nhóm

Đơn giản và phá bế tắc chắc chắn, nhưng hủy toàn bộ công việc đã thực hiện và có thể ảnh hưởng dữ liệu dùng chung.

Dừng từng tiến trình

Chọn một nạn nhân, giải phóng tài nguyên rồi chạy lại Detection; lặp đến khi hết chu trình. Ít lãng phí hơn nhưng tốn nhiều lần kiểm tra.

Tiến trình bị kết thúc phải được dọn tài nguyên, hủy hoặc hoàn tất giao dịch, và đưa dữ liệu về trạng thái nhất quán.



Chọn nạn nhân theo chi phí hệ thống

Ưu tiên và loại tiến trình

Tiến trình hệ thống, tương tác hoặc thời gian thực thường có chi phí dừng cao hơn job nền.

Tiến độ

Xét thời gian đã chạy, thời gian còn lại và lượng kết quả sẽ bị mất khi rollback.

Tài nguyên

Số và loại tài nguyên đang giữ, tài nguyên còn cần và mức độ chúng giải phóng được vòng chờ.

Ảnh hưởng dây chuyền

Số tiến trình phải kết thúc, quan hệ cha–con, giao dịch và người dùng bị tác động.



Chiếm đoạt tài nguyên cần đi kèm rollback



Chọn nạn nhân có chi phí thấp, thu hồi tài nguyên có thể chiếm đoạt hoặc kết thúc tiến trình, khôi phục checkpoint nhất quán rồi kiểm tra lại bể tắc.



Khắc phục lặp lại có thể gây chết đói

Nguy cơ

Một tiến trình có chi phí thấp có thể liên tục bị chọn làm nạn nhân và không bao giờ hoàn thành.

Tính cả lịch sử

Tăng chi phí nạn nhân theo số lần rollback hoặc thời gian chờ; tiến trình từng bị chọn sẽ dần được bảo vệ.

Checkpoint hợp lý

Lưu trạng thái định kỳ và giới hạn số lần khởi động lại để giảm công việc mất và bảo đảm tiến triển.

Mục tiêu khắc phục không chỉ phá bế tắc hiện tại mà còn tránh tái tạo cùng chu trình hoặc hy sinh mãi một tiến trình.



So sánh các phương án khắc phục

Dừng tất cả

Nhanh, chắc chắn; mất nhiều công việc và có thể gián đoạn dịch vụ lớn.

Dừng từng tiến trình

Giảm thiệt hại nhưng cần chọn nạn nhân và chạy Detection nhiều lần.

Chiếm đoạt tài nguyên

Giữ tiến trình sống nhưng chỉ khả thi với tài nguyên có thể thu hồi và trạng thái có thể khôi phục.

Rollback

Bảo toàn tính nhất quán nhờ checkpoint; cần bộ nhớ lưu trạng thái và cơ chế tái thực thi an toàn.

Tổng kết Chương 5



Bế tắc là trạng thái chờ khép kín của một nhóm tiến trình

Bốn điều kiện cần: Độc quyền, giữ và chờ, không chiếm đoạt, chờ vòng tròn

Ngăn chặn phá một điều kiện; tránh bế tắc chỉ cấp phát khi trạng thái vẫn an toàn

Banker dùng Available, Max, Allocation và Need; Safety tìm một chuỗi hoàn thành

Phát hiện dùng wait-for graph hoặc thuật toán ma trận với Request hiện tại

Khắc phục bằng kết thúc tiến trình, chiếm đoạt tài nguyên và rollback



Câu hỏi và bài tập thảo luận

Vì sao có chu trình trong đồ thị phân phối tài nguyên chưa đủ kết luận bế tắc khi tài nguyên có nhiều thể hiện?

Hãy chứng minh thứ tự tài nguyên tăng dần loại bỏ điều kiện chờ vòng tròn.

Trong hệ thống nào nên phát hiện–khắc phục thay vì ngăn chặn hoặc tránh bế tắc?

Email: minhnl@utc.edu.vn